

Всё что вы хотели знать про ACL в Active Directory

 habr.com/ru/articles/809485

April 22, 2024

```
<Sacl>...</Sacl>
<Dacl>
  <AclRevision>4</AclRevision>
  <ACE>
    <AceType TypeName="ACCESS_ALLOWED_OBJECT_ACE_TYPE">5</AceType>
    <AceFlags>...</AceFlags>
    <AceData>
      <AccessMask>
        <Data>10 00 00 00</Data>
        <Bits>
          <b00 Meaning="RIGHT_DS_CREATE_CHILD">0</b00>
          <b01 Meaning="RIGHT_DS_DELETE_CHILD">0</b01>
          <b02 Meaning="RIGHT_DS_LIST_CONTENTS">0</b02>
          <b03 Meaning="RIGHT_DS_WRITE_PROPERTY_EXTENDED">0</b03>
          <b04 Meaning="RIGHT_DS_READ_PROPERTY">1</b04>
          <b05 Meaning="RIGHT_DS_WRITE_PROPERTY">0</b05>
          <b06 Meaning="RIGHT_DS_DELETE_TREE">0</b06>
          <b07 Meaning="RIGHT_DS_LIST_OBJECT">0</b07>
          <b08 Meaning="RIGHT_DS_CONTROL_ACCESS">0</b08>
          <b09>0</b09>
          <b10>0</b10>
          <b11>0</b11>
          <b12>0</b12>
          <b13>0</b13>
          <b14>0</b14>
          <b15>0</b15>
          <b16 Meaning="DELETE">0</b16>
          <b17 Meaning="READ_CONTROL">0</b17>
          <b18 Meaning="WRITE_DAC">0</b18>
          <b19 Meaning="WRITE_OWNER">0</b19>
          <b20 Meaning="SYNCHRONIZE">0</b20>
          <b21 Meaning="RESERVED">0</b21>
          <b22 Meaning="RESERVED">0</b22>
          <b23 Meaning="RESERVED">0</b23>
          <b24 Meaning="ACCESS_SYSTEM_SECURITY">0</b24>
          <b25 Meaning="MAXIMUM_ALLOWED">0</b25>
          <b26 Meaning="RESERVED">0</b26>
          <b27 Meaning="RESERVED">0</b27>
          <b28 Meaning="GENERIC_ALL">0</b28>
          <b29 Meaning="GENERIC_EXECUTE">0</b29>
          <b30 Meaning="GENERIC_WRITE">0</b30>
          <b31 Meaning="GENERIC_READ">0</b31>
        </Bits>
      </AccessMask>
      <Flags>...</Flags>
      <ObjectType Name="Account Restrictions">4C164200-20C0-11D0-A768-00AA006E0529</ObjectType>
      <InheritedObjectType Name="inetOrgPerson">4828CC14-1437-45BC-9B07-AD6F015E5F28</InheritedObjectType>
      <SID CommonName="NONE_MAPPED" StringRepresentation="S-1-5-32-554">...</SID>
    </AceData>
  </ACE>
</ACE> </ACE>
```

Введение

В данной статье я постарался максимально полно и глубоко рассказать про построение и внутреннее использование ACL (Access Control List) внутри Active Directory. В этой статье нет рассказов про "null DACL" и "empty DACL" и тому подобного. Если читатель хочет изучить все более простые нюансы использования ACL, то рекомендую почитать [другую мою статью](#) или лучше сразу почитать комментарии к моему [тестовому коду для этой статьи](#).

Что будет в этой статье:

- Расскажу про все 22 различных типа ACE, а также разделю их на 4 различных вида;

- Расскажу, почему прав вида "GENERIC" не существует;
- Покажу, как именно флаги из ACCESS_MASK работают при проверках в Active Directory;
- Расскажу почему вы не сможете "сделать RBCD" имея AllExtendedRights на "computer";
- Расскажу и дам ссылку на программу для получения всех "control access rights" (extended rights, validated writes, property sets);
- Покажу, как получить полный список всех атрибутов, связанных control access rights и подчинённых классов для любого объекта в домене;
- Расскажу про каждое "validated write" в отдельности и покажу как можно обойти их контроль;
- Как именно хранятся security descriptors в NTDS.DIT и почему их там мало;
- Дам таблицу для всех "extended access rights" со ссылками на алгоритмы их использования;

Статья предполагает, что у читателя уже есть какие-то знания по построению ACL. Никаких "основ" в данной статье нет, только hardcore.

Виды ACE

На самом деле существует 22 различных типа ACE, вроде ACCESS_ALLOWED_ACE_TYPE или ACCESS_ALLOWED_CALLBACK_OBJECT_ACE_TYPE. Но их можно объединить во всего лишь 4 общих вида "access control entity".

Первый вид ACE объединяет следующие типы: ACCESS_ALLOWED_ACE_TYPE, ACCESS_DENIED_ACE_TYPE, SYSTEM_AUDIT_ACE_TYPE, SYSTEM_MANDATORY_LABEL_ACE_TYPE и SYSTEM_SCOPED_POLICY_ID_ACE_TYPE. В данном виде ACE можно устанавливать права только на основе ACCESS_MASK и только для субъекта, определяемого посредством SID. Этот вид ACE является наиболее простым и наиболее часто используемым.

Второй вид ACE объединяет следующие типы: ACCESS_ALLOWED_OBJECT_ACE_TYPE и ACCESS_DENIED_OBJECT_ACE_TYPE. В данном виде ACE права можно устанавливать как на основе ACCESS_MASK, так и на основе использования "расширенных прав" на основе GUID. Кроме того возможно устанавливать права на типы объектов, которые могут создаваться как подчинённые

к текущему объекту (inherited). Данный вид ACE наиболее часто используется в Active Directory.

Третий вид ACE объединяет следующие типы:

ACCESS_ALLOWED_CALLBACK_ACE_TYPE, ACCESS_DENIED_CALLBACK_ACE_TYPE, SYSTEM_AUDIT_CALLBACK_ACE_TYPE и SYSTEM_RESOURCE_ATTRIBUTE_ACE_TYPE. Данный вид является, как бы, "расширением" вида №1: здесь права доступа также определяются посредством использования ACCESS_MASK и SID, однако в данном виде добавляется использование "conditional expressions". Использование "conditional expressions" позволяет строить логические правила, которые будут определять, применяется ли текущее правило к данному конкретному access token или нет. Например, можно добавить правило типа ACCESS_ALLOWED_CALLBACK_ACE_TYPE и дать в нём права все права на какой-то объект для SID, который представляет какую-то группу, и затем в "conditional expression" задать дополнительное ограничение для доступа только с определённых компьютеров (ограничение с применением так называемых "device claims").

Четвёртый вид ACE объединяет следующие типы:

ACCESS_ALLOWED_CALLBACK_OBJECT_ACE, ACCESS_DENIED_CALLBACK_OBJECT_ACE, SYSTEM_AUDIT_CALLBACK_OBJECT_ACE и SYSTEM_AUDIT_OBJECT_ACE_TYPE. Здесь всё также, как и во втором виде: в дополнение к ACCESS_MASK можно использовать также и "расширенные" права на основе GUID. Но в четвёртом виде в дополнение ко всем возможностям второго вида тут, также как и в третьем виде ACE, можно применять "conditional expressions". Однако необходимо сказать, что на самом деле в четвёртом виде ACE возможность работы с "conditional expressions" отсутствует. То есть "conditional expressions" можно создавать, однако все функции проверки их не учитывают.

Почему прав вида "GENERIC" не существует

Основные права доступа для того или иного ACE определяются посредством поля ACCESS_MASK, представляющего собою 32-х битное значение, каждый бит которого устанавливает то или иное право доступа. И можно увидеть, что в описании стандартных битов доступа в ACCESS_MASK присутствуют биты GENERIC_READ, GENERIC_WRITE и GENERIC_ALL. Однако дело в том, что на самом деле внутри security descriptor не может быть никаких ACE с какими-либо установленными GENERIC_* битами. Почему же так происходит, и зачем на самом деле нужны биты GENERIC_*? Дело в том, что существует два вида операций с security descriptor, при котором система использует ACCESS_MASK. Первая операция это собственно добавление в security descriptor новых ACE. Установка GENERIC битов при создании

новых ACE запрещена: [в данном документе](#) описывается, что установка данных битов может привести к "unintended results" (непредвиденным результатам). Да и установка этих битов, скорее всего, пройдет незамеченной: данные биты не учитываются в алгоритме проверки авторизации (https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-dtyp/4f1bbcbb-814a-4c70-a11e-2a5b8779a6f9 - псевдокод для алгоритма проверки авторизации). Второй же операцией является проверка прав доступа на основании какого-либо access token текущего security descriptor. И при второй операции существует понятие "отображения" прав вида GENERIC на набор битов, которые обычно являются битами из object-specific rights. То есть в функциях проверки прав доступа, вроде AccessCheckByTypeResultListAndAuditAlarmByHandle, есть отдельный параметр GenericMapping, который определяет, что на какую комбинацию битов будут "отображаться" биты GENERIC. Напрямую использовать биты GENERIC в параметре DesiredAccess нельзя - система просто не будет их учитывать в алгоритме проверки доступа.

Таким образом, получается, что несмотря на то, что во многих источниках приводят примеры, что "кто-то имеет права GENERIC_ALL на какой-то объект", на самом деле этот "кто-то" имеет просто набор битов из "object-specific rights", которые система стандартно "отображает" на GENERIC_ALL. Например для объектов из Active Directory "отображения" прав типа GENERIC определяются в [этом документе](#). Для объектов из обычной файловой системы это уже будут другие комбинации битов, для прав на сервисы - третьи комбинации битов, и так далее. Кстати, я в своей библиотеке постарался документировать, какие именно биты означают для каждого из типов объектов, мой список можно [найти тут](#). Для каждого типа объектов биты в ACCESS_MASK будут означать совершенно различные права, и нельзя напрямую поставить бит GENERIC_ALL в ACE для какого-то объекта чтобы получить все-все права на его использование.

Флаги и control access rights

RIGHT_DS_CREATE_CHILD и RIGHT_DS_DELETE_CHILD: в качестве object GUID выступает schemaID для компонента, который может создавать принципал в качестве подчинённого текущему (то есть если текущий объект это OU=Computers, а objectGUID=computer, то для SID из ACE это означает, что этот SID может создавать/удалять только объекты типа "computer" внутри OU=Computers). Если в текущем ACE нет элемента objectGUID, то это означает, что данному SID разрешено создавать/удалять подчинённые объекты любых типов.

RIGHT_DS_LIST_CONTENTS: просто выдаёт право какому-то SID на просмотр подчинённых объектов. Параметр objectGUID никак не влияет, однако потенциально можно применять RIGHT_DS_LIST_CONTENTS вместе с "conditional expression".

RIGHT_DS_WRITE_PROPERTY_EXTENDED: в качестве objectGUID используется значение для одного из validated writes. Однако на самом деле в этом случае objectGUID может быть абсолютно любым, при проверке влияет ли имя того атрибута, который пользователь пытается изменить.

RIGHT_DS_READ_PROPERTY и RIGHT_DS_WRITE_PROPERTY: в качестве objectGUID используется либо property set, либо schemaID для отдельно property конкретного объекта. Если в текущем ACE нет элемента objectGUID, то это означает, что данному SID разрешено читать/писать по отношению ко всем свойствам данного объекта.

RIGHT_DS_DELETE_TREE: влияет только в случае, когда идёт попытка удалить всё дерево объектов (головной объект вместе со всеми его дочерними объектами).

RIGHT_DS_LIST_OBJECT: определяет права на видимость конкретно этого объекта для выбранного SID. Параметр objectGUID никак не влияет, однако потенциально можно применять RIGHT_DS_LIST_CONTENTS вместе с "conditional expression". Использование данного флага имеет смысл только в случае использования специального режима "List Object", который устанавливается отдельным битом в dSHeuristics (https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/4a7705f7-c61e-4020-86a7-41a44fb233e5).

RIGHT_DS_CONTROL_ACCESS: в качестве objectGUID используется "extended rights". Если в ACE элемент objectGUID отсутствует, то это означает, что данный SID имеет все "extended rights".

Что означают "control access rights" и почему вы не сможете "сделать RBCD" имея AllExtendedRights на "computer"

Есть следующие типы "control access rights": - Extended Rights; - Validated Writes; - Property Sets;

Extended Rights это права, которые проверяются в каких-то определённых алгоритмах. Вроде "если пользователь запрашивает синхронизацию домена, то у него должны быть права на синхронизацию". То есть extended rights это права на определённые действия. Кстати, ниже в статье я привожу все найденные мною ссылки на описание алгоритмов, где используются все extended rights.

Validated Writes представляют собой "расширенные права" модификации определённых атрибутов. То есть они работают только для определённого набора атрибутов (ниже в статье я расскажу более подробно) и только при их модификации.

Property Set представляют собой право доступа к набору атрибутов объекта. Объединение атрибутов в наборы позволяет избежать увеличения размеров ACL

(вместо кучи однотипных ACE в ACL добавляется только один элемент). Права на property set это права на модификацию/чтение набора атрибутов. Ниже я дам ссылку на программу, с помощью которой можно получить полный состав атрибутов для всех property sets.

Почему же нельзя записать в атрибут msDS-AllowedToActOnBehalfOfOtherIdentity если у вас есть все extended rights на computer? Ещё раз: extended rights это права на определённые действия. А запись в атрибуты контролируется либо отдельными ACE с правами записи для нужного нам атрибута, либо получением прав на соответствующий property set (в нашем случае это "User-Account-Restrictions", GUID=4c164200-20c0-11d0-a768-00aa006e0529).

Наследование правил доступа

Практически каждый элемент в Active Directory является частью какого-то "дерева", то есть имеет "предка" и "наследников". И для возможности автоматического наследования применяются "inherited ACEs": ACE с установленными флагами OBJECT_INHERIT_ACE, CONTAINER_INHERIT_ACE, NO_PROPAGATE_INHERIT_ACE, INHERIT_ONLY_ACE и INHERITED_ACE. В случае, если ACE имеет простые типы (ACCESS_ALLOWED_ACE_TYPE или ACCESS_DENIED_ACE_TYPE) и установлен, например, флаг CONTAINER_INHERIT_ACE, то данное ACE автоматически перейдёт в ACL для нового подчинённого объекта, который будет являться контейнером, причём тип этого контейнера никак не ограничен. Если же используется ACE с указанием GUID (ACCESS_ALLOWED_OBJECT_ACE_TYPE и ACCESS_DENIED_OBJECT_ACE_TYPE), то в этих ACE может присутствовать значение InheritedObjectType. В этом случае наследование этого ACE ограничивается только тем типом, который упомянут в InheritedObjectType. В InheritedObjectType может быть только значение GUID, равное значению атрибута schemaIDGUID из пространства имён "CN=Schema".

Получение всех атрибутов для любого объекта в Active Directory

Для того, чтобы проверить доступ ко всем элементам какого-либо объекта в Active Directory необходимо, прежде всего, получить эти самые элементы. И, на самом деле, вопрос получения всех необходимых для проверки элементов является достаточно интересным.

Для начала найдём все возможные элементы, к которым может быть предоставлен доступ, если мы рассматриваем отдельный объект в Active Directory. Во-первых, доступ может быть предоставлен к самому объекту. Однако на самом деле при проверке доступа учитывается не идентификатор какого-то конкретного объекта, а

GUID его основного класса. Далее в Active Directory есть так называемые "control access rights": специальные права, идентифицируемые отдельными GUID. Также нужно понимать, что каждое "control access right" относится только к определённому набору типов объектов, и при проверке надо найти только необходимые "control access right". Также в Active Directory есть понятие "property sets": один из типов "control access rights", позволяющий выдавать права не просто на отдельные атрибуты объекта, а сразу на набор таких атрибутов. Причём при проверке доступа необходимо учитывать, что каждый "property set" также относится только к определённым типам объектов. Кроме того, так как "property set" может относиться сразу к нескольким типам объектов, то при проверке доступа к определённому объекту необходимо включить только те атрибуты, которые относятся именно к данному типу.

Ещё одним немаловажным моментом является то, что в правилах ACE могут указываться права не только на элементы, относящиеся к текущему типу объекта, но также могут присутствовать и права на подчинённые типы объектов. Так что для получения полной картины доступа необходимо найти всех возможных потомков данного типа.

В итоге для определения полной картины доступа ко всем элементам какого-либо объекта нам будет необходимо получить картину доступа ко всему "дереву" подчинённых элементов. Для построения подобного "дерева" в Windows применяется массив элементов специального типа OBJECT_TYPE_LIST. В данном типе всего два свойства: Level и ObjectType. Свойство Level имеет тип WORD и обозначает "номер уровня" данного элемента в "дереве". Для нужд проверки доступа в Active Directory существует всего три таких уровня: ACCESS_OBJECT_GUID (нулевой уровень, здесь может быть только один элемент), ACCESS_PROPERTY_SET_GUID и ACCESS_PROPERTY_GUID. Свойство ObjectType имеет тип GUID и обозначает идентификатор элемента в схеме Active Directory.

Теперь распишем более подробно, какие именно элементы могут находиться на каждом уровне "дерева":

- ACCESS_OBJECT_GUID

В массиве может быть только один элемент с таким значением Level. Значение ObjectType данного элемента будет соответствовать идентификатору основного типа данного объекта;

- ACCESS_PROPERTY_SET_GUID
 - Все "control access rights" (extended rights, validated writes, property sets);
 - Все атрибуты, не вошедшие ни в один "property set";
 - Все дочерние типы классов;

- ACCESS_PROPERTY_GUID

На данном уровне "дерева" могут находиться только отдельные атрибуты, принадлежащие конкретным "property sets";

Теперь перейдём к алгоритмам получения каждого из типов элементов. Для начала необходимо получить GUID для основного класса текущего объекта. В Active Directory каждый объект имеет набор классов, от которых он наследует свои свойства. Такой набор классов находится в атрибуте "objectClass" любого объекта в AD. Для целей проверки доступа "основным классом" считается тот класс, имя которого стоит последним в атрибуте "objectClass". Для получения GUID "основного класса" необходимо получить значение атрибута schemaIDGUID из пространства имён LDAP "CN=Schema".

Кроме "основного класса" для определённого объекта есть и общий набор классов. То есть на самом деле текущий объект наследует атрибуты из всех классов, упомянутых в "objectClass". А каждый из этих классов будет иметь свой набор родительских классов со своими атрибутами и так далее. Таким образом, для получения всех атрибутов и "control access rights" необходимо последовательно находить все атрибуты и "control access rights", относящиеся к каждому из родительских классов текущего объекта. К каждому такому классу могут относиться свои атрибуты и "control access rights", однако по правилам их свойства наследуются всеми его потомками. Все получившиеся значения записываются в общее "дерево" на соответствующем уровне.

Получение всех "control access rights" я опишу в отдельном разделе данной статьи, так что здесь на этом останавливаться не буду.

Для получения всех возможных потомков "основного класса" необходимо получить значение вычисляемого атрибута "possibleInferiors". Данный атрибут присутствует только в пространстве имён "CN=Schema".

Реализацию алгоритма поиска всех элементов для любого объекта в Active Directory вы можете найти по [этой ссылке](#).

Также хочу добавить, что в Active Directory существует набор атрибутов, которые просто невозможно получить, несмотря на то, какие права вам будут на них выданы.

В число "секретных" входят следующие атрибуты:

- unicodePwd;
- dBCSPwd;
- currentValue;
- initialAuthIncoming;
- initialAuthOutgoing;
- lmPwdHistory;
- ntPwdHistory;
- priorValue;
- supplementalCredentials;
- trustAuthIncoming;
- trustAuthOutgoing;
- msDS-ExecuteScriptPassword;
- pekList;

Значение подобных атрибутов всё-таки можно получить, но только путём прямой работы с "базой данных" - файлом NTDS.DIT.

Общее правило проверки ACE

Общие правила проверки ACE описаны в [официальной документации](#) от Microsoft. Ниже я даю более простое описание этих же правил.

Первоначально условимся, что для каждого элемента дерева из "object list" существует два отдельных значения с типом DWORD: Deny(v) и Grant(v). В начале выполнения проверки оба этих значения для всех узлов устанавливаются в 0. Корневой узел "object list" будем обозначать root.

Далее проверяем все ACE последовательно. Условимся, что поле ACCESS_MASK для каждого ACE будем обозначать M.

Если текущее ACE имеет тип ACCESS_ALLOWED_ACE, то устанавливаем значение Allow(root) как $\text{Allow}(\text{root}) \leftarrow \sim \text{Deny}(\text{root}) \& M$ (то есть устанавливаем в Allow(root) только те биты из M, которые не установлены в Deny(root)). Далее устанавливаем

Allow(v) для каждого подчинённого узла v. Правила установки Allow(v) такие же, как и для Allow(root) (то есть устанавливаются только те биты из M, которые отсутствуют в Deny(v)).

Если текущее ACE имеет тип ACCESS_DENIED_ACE, то добавляем значение M в Deny(root), а затем в Deny(v) для каждого потомка v.

Если текущее ACE является ACCESS_ALLOWED_OBJECT_ACE и у него отсутствует значение в поле ObjectType, то приравниваем данное ACE к ACCESS_ALLOWED_ACE и выполняем соответствующие ACCESS_ALLOWED_ACE действия.

Если текущее ACE является ACCESS_ALLOWED_OBJECT_ACE и у него установлено значение в поле ObjectType, однако это значение не соответствует ни одному GUID из "object list", то пропускаем данное ACE.

Если текущее ACE является ACCESS_ALLOWED_OBJECT_ACE и у него установлено значение в поле ObjectType, которое соответствует какому либо GUID из "object list" то устанавливаем значение Allow(v) для текущего узла и всех его подчинённых узлов по общему правилу $Allow(v) \leftarrow \sim Deny(v) \& M$. Если на каком-то уровне "object list" получится, что все значения Allow(v) равны, то тогда добавляем значение Allow(v) в Allow(p), где "p" является корневым узлом для текущего уровня. Другими словами если у какого-то узла правила Allow для всех его листьев одинаковы, то тогда это правило переносится в Allow для самого корневого узла.

Если текущее ACE является ACCESS_DENIED_OBJECT_ACE и у него отсутствует значение в поле ObjectType, то приравниваем данное ACE к ACCESS_DENIED_ACE и выполняем соответствующие ACCESS_DENIED_ACE действия.

Если текущее ACE является ACCESS_DENIED_OBJECT_ACE и у него установлено значение в поле ObjectType, однако это значение не соответствует ни одному GUID из "object list", то пропускаем данное ACE.

Если текущее ACE является ACCESS_DENIED_OBJECT_ACE и у него установлено значение в поле ObjectType, и оно соответствует одному из GUID из "object list", то добавляем значение M во все Deny для всех подчинённых и родительских узлов.

Проверка ACE с "control access rights"

Основной [документ](#) от Microsoft. Ниже я даю более простое описание этих же правил.

Основным условием для отнесения ACE к "control access rights" является наличие установленного бита RIGHT_DS_CONTROL_ACCESS в ACCESS_MASK.

Если ACE является ACCESS_ALLOWED_OBJECT_ACE и поле ObjectType отсутствует, то это означает, что для данного SID разрешены все "control access rights".

Если ACE является ACCESS_ALLOWED_OBJECT_ACE и поле ObjectType содержит какой-то GUID, то права у данного SID есть только на "control access right" с соответствующим GUID.

Если ACE является ACCESS_DENIED_OBJECT_ACE и поле ObjectType отсутствует, то это означает, что для данного SID запрещены все "control access rights".

Если ACE является ACCESS_DENIED_OBJECT_ACE и поле ObjectType содержит какой-то GUID, то права у данного SID отсутствуют (явно) только на "control access right" с соответствующим GUID.

Проверка ACE с "validated write"

Основной [документ](#) от Microsoft. Ниже я даю более простое описание этих же правил.

Основным условием для отнесения ACE к "validated write" является наличие установленного бита RIGHT_DS_WRITE_PROPERTY_EXTENDED. Права типа "validated write" являются "дополнительными", и проверяются только если для данного SID ранее отсутствовали ACE явно разрешающие или запрещающие модификацию соответствующих атрибутов объекта.

Если ACE является ACCESS_ALLOWED_OBJECT_ACE и поле ObjectType отсутствует, то это означает, что данный SID имеет право на все "validated write".

Если ACE является ACCESS_ALLOWED_OBJECT_ACE и поле ObjectType содержит какой-то GUID, то данный SID имеет право только на "validated write" с соответствующим GUID - так написано в документации. На деле же GUID для "validated write" может быть произвольным, само правило "validated write" сработает только при модификации соответствующего атрибута соответствующего объекта.

Если ACE является ACCESS_DENIED_OBJECT_ACE и поле ObjectType отсутствует, то это означает, что данный SID не имеет никаких "validated write".

Если ACE является ACCESS_DENIED_OBJECT_ACE и поле ObjectType содержит какой-то GUID, то это означает, что для данного SID запрещено только одно конкретное "validated write".

Алгоритм получения всех "control access rights"

В "control access rights" Microsoft включает все три следующих группы: extended access rights, validated writes, а также property sets. Под "extended access rights"

понимается GUID, обозначающий дополнительное право доступа, которое может быть проверено системой при определённых условиях. Более подробно про все "extended access rights" я напишу позже в этой статье. Под "validated writes" понимается специальное право на "проверяемую" модификацию определённых атрибутов внутри ActiveDirectory. Под "property sets" понимается некий GUID, который позволяет давать права на чтение или запись сразу для группы атрибутов.

Все "control access rights" можно получить из LDAP для текущего домена. Необходимо сказать, что набор "control access rights" не является фиксируемым и в будущем он может быть изменён. Для нахождения всех "control access rights" во-первых необходимо выполнить LDAP запрос с фильтром "rightsGuid=*" по отношению к "configuration namespace" (это пространство имён начинается с "CN=Configuration"). В этом запросе нам будут нужны следующие атрибуты:

- name (для короткого наименования "control access right");
- displayName (для полного описания "control access right");
- appliesTo (для получения всех атрибутов, к которым применяется текущее "extended right");
- validAccesses (для получения типа "control access right");
- rightsGuid (для получения уникального идентификатора "control access right");

Для получения информации о том, какие именно атрибуты включаются в тот или иной "property set" необходимо выполнить запрос к "schema namespace" (имя начинается на "CN=Schema") с LDAP фильтром "schemaldGuid=*". Если у атрибута есть элемент с именем "attributeSecurityGUID", то его значение будет соответствовать rightsGuid для определённого "property set".

Программу для получения списка всех "control access rights" из Active Directory вы можете найти по [этой ссылке](#).

Про Validated Writes

Права вида "Validated Writes" выделяются путём включения в access mask флага RIGHT_DS_WRITE_PROPERTY_EXTENDED. В Windows Server 2019 существуют следующие вида Validated Writes: • Self-Membership; • Validated-DNS-Host-Name; • Validated-SPN; • Validated-MS-DS-Behavior-Version; • Validated-MS-DS-Additional-DNS-Host-Name; • DS-Validated-Write-Computer;

Self-Membership

Проверяет запись в атрибут "member" для объектов типа "group". Для прохождения проверки необходимо, чтобы выполнялась операция либо добавления, либо удаления, чтобы добавлялось/удалялось единственное значение и чтобы это значение совпадало с SID пользователя, от имени которого выполняется данная операция.

Validated-DNS-Host-Name

Проверяется запись в атрибут "dnsHostName". Проверка выполняется только для класса "computer" и его наследников. Проверяется то, что новое значение атрибута будет являться объединением значения атрибута "sAMAccountName" (без последнего "\$") и имени текущего домена (вроде "computer.domain.lan").

Validated-SPN

Проверяется запись в атрибут "servicePrincipalName". Проверка выполняется только для класса "computer" и его наследников. Проверяется, что новое значение:

- Состоит только из двух частей;
- Последняя часть совпадает с одним из следующих атрибутов: dnsHostName, samAccountName или msDS-AdditionalDnsHostName;

Validated-MS-DS-Behavior-Version

Проверяет запись в атрибут "msDS-Behavior-Version". Возможна модификация значения только для классов "domainDNS" и "crossRefContainer". Новое значение этого атрибута должно быть больше текущего. Новое значения для класса "domainDNS" должно быть меньшим, либо эквивалентным значениям атрибутов "msDS-Behavior-Version" для всех классов типа "nTDSA" в текущем домене.

Validated-MS-DS-Additional-DNS-Host-Name

Проверяется запись в атрибут "msDS-AdditionalDnsHostName". Проверка выполняется только для класса "computer" и его наследников. Проверяется, что окончание нового значения совпадает с разрешёнными суффиксами DNS (значение текущего DNS суффикса + атрибут msDS-AllowedDNSSuffixes).

DS-Validated-Write-Computer

Проверяет запись в атрибут "msDS-KeyCredentialLink". Проверка выполняется только для класса "computer" и его наследников. Каких-то полных данных что именно

проверяется и как найти не удалось.

В процессе исследования выяснились две особенности, относящиеся к Validated Writes. Первая особенность состоит в том, что на самом деле правило Validated Write будет выполняться только в том случае, когда у вас нет никаких других прав на данный атрибут. То есть если у вас есть ACE явно разрешающее доступ к атрибуту servicePrincipalName, то вы сможете записать в него всё, что вам захочется. Вторая особенность состоит в том, что если в ACE установлен флаг RIGHT_DS_WRITE_PROPERTY_EXTENDED, то нет никакой разницы какой именно GUID стоит в поле ObjectType. Так или иначе правило Validated Write просто будет проверять в какой атрибут вы собираетесь писать и в зависимости от имени этот атрибута будет вызывать соответствующее правило. Как следствие - если у вас есть право на Validated Write для какого-то одного атрибута, то вы сможете писать в любой атрибут, который проверяет любое из Validated Write.

Работа с security descriptor внутри NTDS.DIT

Наверняка многие знают, что значение "security descriptor" для каждого объекта хранится в специализированном атрибуте "nTSecurityDescriptor". Наверное, некоторые даже знают, что этот атрибут является "вычисляемым", то есть для его получения необходимо выполнить отдельный LDAP запрос с указанием только этого атрибута в качестве результирующего. Но почему же "nTSecurityDescriptor" как-то "вычисляется" внутри Active Directory?

Изначально все security descriptors действительно хранились в отдельном атрибуте, связанном с каждым объектом в Active Directory. Однако такой способ хранения привёл к чрезмерным размерам базы данных AD. Почему же это произошло? Дело в том, что в Active Directory объекты обычно входят в состав какого-то "дерева" объектов, где всегда есть "родитель" и "наследники". И если, например, изменяется одно ACE в "родительском" security descriptor, то оно должно (если имеет соответствующие флаги) быть добавлено также в security descriptors для всех "наследников". Предположим, что меняется одно ACE для корневого элемента домена, в котором миллион элементов. Если один ACE, в среднем, имеет размер в 15Кб, то внесение его во все security descriptors для всех наследников приведёт к увеличению общей базы данных на более чем 1Гб.

Однако было предложено простое решение: выделить все security descriptors в отдельную таблицу базы данных. Это оказалось гораздо более выгодно так как в Active Directory очень много однотипных элементов с одинаковыми правами, а также потому, что изменение прав на объекты обычно выполняются достаточно редко. Например, у меня в тестовом домене таблица "datatable" из NTDS.DIT содержит 3764 элемента, тогда как количество всех security descriptors равно всего 120. И причём из 120-ти SD есть всего 2 SD, которые в сумме используются в 3111 элементах. То есть

82 процента всех элементов в моём тестовом домене используют всего 2 стандартных SD. А из всех 120-ти SD есть 72, которые используются только в одном элементе домена. Оставшиеся 46 SD используются в разном количестве элементов, но ни для одного из этих SD количество использующих его элементов не превышает 100.

Внутри NTDS.DIT за хранение security descriptors отвечает таблица "sd_table". Внутри этой таблицы есть следующие поля:

- sd_hash - значение MD5 для всего бинарного значения security descriptor;
- sd_id - идентификатор данного security descriptor (именно это значение на самом деле хранится в атрибуте nTSecurityDescriptor);
- sd_refcount - количество элементов Active Directory, в которых используется данный security descriptor;
- sd_value - собственно бинарное значение security descriptor;

Также в NTDS.DIT есть ещё одна таблица - "sdproptable". Информация в данной таблице нужна для осуществления "propagation" (распространения) изменений в security descriptor. Похоже, что информация в неё записывается только по мере необходимости так как в моём тестовом NTDS.DIT таблица "sdproptable" пустая. Предположу, что в Active Directory существует какой-то отдельный процесс, который запускает изменение "дочерних" SD при изменении в "родительском" SD.

Информация по всем "extended rights"

Ниже я привожу таблицу, где я постарался для каждого из "extended rights" найти все возможные ссылки на использование. Таким образом, можно понять, где именно и в каких случаях проверяется наличие каждого из "расширенных прав". Приведены ссылки только на официальную документацию Microsoft.

Extended Right Common Name	Useful Links To Rights Usage
Domain-Administer- Server	

User-Change-Password

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/6e803168-f140-4d23-b2d3-c3a8ab5917d2

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/14904966-0dc2-42ae-bc99-4f474ebb9613

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/c714e48c-ea21-48b0-913d-fc065ab3dda3

User-Force-Change-Password

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/6e803168-f140-4d23-b2d3-c3a8ab5917d2

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/14904966-0dc2-42ae-bc99-4f474ebb9613

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/c714e48c-ea21-48b0-913d-fc065ab3dda3

Send-As

Receive-As

Send-To

Open-Address-Book

DS-Replication-Get-
Changes

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/4abd4c7c-c078-4809-95bc-38e657d5c034

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/15bbda52-7742-4ce3-9327-018647b0fc26

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/d6b87d97-2b74-4715-b51f-564628181e3e

DS-Replication-
Synchronize

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/d3d19d15-8427-4d4d-8256-d5fb11333292

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/5d8570a6-cb92-4e37-ad0d-6d32cd2cdc0d

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/cf77e5b5-bc22-48fe-8138-da20c2514a52

DS-Replication-Manage-
Topology

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/caa21197-11be-4efe-b9ae-35477ee68c8c

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/eef0b04d-0d0e-407d-8b69-4af8c1e5each

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/5339140a-621e-4a3b-8230-b487687be881

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/758591d7-08b6-4f5a-b78a-f057633155b9

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/6dd33731-847a-456d-a0e3-6d3e360c978a

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/022f186b-1b1c-434f-899f-c9170f787b56

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/574fd22a-501e-43fd-ad9c-0b07830e7913

Change-Schema-Master

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/0645718c-4715-45c0-986f-53c1340af9ef

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/dc21d519-c3e1-4ce0-b061-cc2beb95db5a

https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/a21db735-6025-4244-9cfe-6ce6582114a8

Change-Rid-Master	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/dc21d519-c3e1-4ce0-b061-cc2beb95db5a https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/5aa61b35-6147-46f6-b3bb-d5757ed9347a https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/ae92addd-470c-4b12-92b8-ccaf18b19560
Do-Garbage-Collection	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/639b5050-a896-4bc7-8239-41cfb0e9d244 https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/d05e3801-9cdc-459d-b37b-fb4e09baef7e https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/0341ab73-f5d1-4b13-9b16-c7b72af98f3f https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/bca65613-d36c-429e-8464-5517e7a9fcfc
Recalculate-Hierarchy	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/98f87fb7-9d35-4c9a-bed9-edc30ba8cc22
Allocate-Rids	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/84390cb7-9eb6-4b85-8975-935f8153c78c

Change-PDC	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/d1e7ae7c-02f7-479b-b55a-986911b7a896 https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/dc21d519-c3e1-4ce0-b061-cc2beb95db5a
Add-GUID	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/7dfcb38c-3cb9-4215-ae1c-ef209fd251ae
Change-Domain-Master	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/6d9188ff-1f93-4a0c-af52-f693af77c96f https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/dc21d519-c3e1-4ce0-b061-cc2beb95db5a
msmq-Receive-Dead-Letter	
msmq-Peek-Dead-Letter	
msmq-Receive-computer-Journal	
msmq-Peek-computer-Journal	
msmq-Receive	
msmq-Peek	
msmq-Send	

msmq-Receive-journal	
msmq-Open-Connector	
Apply-Group-Policy	
DS-Install-Replica	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/a2ad3e15-b839-49dc-9fa2-ee81b85bdcb1
Change-Infrastructure-Master	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/56719fea-1c0b-4fee-bd68-0bfa557fc8d7 https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/dc21d519-c3e1-4ce0-b061-cc2beb95db5a
Update-Schema-Cache	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/1c672104-f8c2-45f5-889f-f7738dcb8356 https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/9d2511b0-ba65-40a9-b962-436e3b23d387
Recalculate-Security-Inheritance	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/ddc8da4a-6ac8-4193-b51c-205cebbf483b
DS-Check-Stale-Phantoms	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/f7dd6c52-b586-4764-afcf-19764ab2f0c8
Certificate-Enrollment	TOO MANY LINKS
Generate-RSoP-Planning	

Refresh-Group-Cache	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/b08c08ac-6186-4973-afd6-9a5c9229d792
SAM-Enumerate-Entire-Domain	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/6bdc92c0-c692-4ffb-9de7-65858b68da75
Generate-RSoP-Logging	
Create-Inbound-Forest-Trust	
DS-Replication-Get-Changes-All	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/4abd4c7c-c078-4809-95bc-38e657d5c034 https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/15bbda52-7742-4ce3-9327-018647b0fc26
Migrate-SID-History	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/fbc94975-28ef-4334-bb47-35708a15d586
Reanimate-Tombstones	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/76df255d-dc67-4c1f-adc6-1e5b60021304
Allowed-To-Authenticate	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-kile/2e5dcf34-4b51-44a0-b45a-277ed616ca39
DS-Execute-Intentions-Script	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/5525ac5d-3ac8-4115-8b60-6d14308f41ef

DS-Replication-Monitor-Topology	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/caa21197-11be-4efe-b9ae-35477ee68c8c https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/e6685d31-5d87-42d0-8a5f-e55d337f47cd
Update-Password-Not-Required-Bit	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/a2ad3e15-b839-49dc-9fa2-ee81b85bdcb1
Unexpire-Password	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/c714e48c-ea21-48b0-913d-fc065ab3dda3 https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/9a014fcd-83f6-4432-8110-31fae3d78f6b https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/22b79fa8-13bd-4952-a399-9d5a93b2256f https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/a2ad3e15-b839-49dc-9fa2-ee81b85bdcb1
Enable-Per-User-Reversibly-Encrypted-Password	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-samr/a2ad3e15-b839-49dc-9fa2-ee81b85bdcb1
DS-Query-Self-Quota	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/fc133ca8-e58f-4be4-b129-4fa3b5cdcfa2 https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/e6685d31-5d87-42d0-8a5f-e55d337f47cd

Read-Only-Replication-Secret-Synchronization	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/d3d19d15-8427-4d4d-8256-d5fb11333292 https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/78bca39e-cba0-4127-95fe-7e2afdad0101 https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/5072bd62-1051-4f45-a87b-b06f92236b43 https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/e6685d31-5d87-42d0-8a5f-e55d337f47cd
Reload-SSL-Certificate	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/4cf26e43-ae0b-4823-b00c-18205ab78065
DS-Replication-Get-Changes-In-Filtered-Set	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/4abd4c7c-c078-4809-95bc-38e657d5c034 https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/15bbda52-7742-4ce3-9327-018647b0fc26
Run-Protect-Admin-Groups-Task	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/f9dbd527-5594-4d27-be4b-ec16d23136e6
Manage-Optional-Features	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/3177e885-8912-471b-8fa5-761cabd6b3ff https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/8106371b-9d69-413a-8e01-7fd97eca81b1

DS-Clone-Domain-Controller	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-drsr/9b8b5527-29c2-4857-ae8a-d689e7577569
Certificate-AutoEnrollment	TOO MANY LINKS
DS-Set-Owner	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/7afacb02-548b-4e74-bf96-04b0bf0c71b6
DS-Bypass-Quota	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/3f21a865-6ebe-4883-8f10-2207a8628b86
DS-Read-Partition-Secrets	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/e6685d31-5d87-42d0-8a5f-e55d337f47cd
DS-Write-Partition-Secrets	https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/c714e48c-ea21-48b0-913d-fc065ab3dda3
Domain-Password	